claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_d88615a4-f56\agent-a646f11db19db3854.jsonl`
- SHA-256: `66b04398c7b18b4efd4c37c8cf08335271510a176c64228d0e9654baff703fa7`
- Source modified: `2026-06-07T09:11:47+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_d88615a4-f56`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`

Transcript

1. user (2026-06-07T09:10:30.801Z)

Read these files in C:/Users/avidu/Projects/badminton-highlight-indexer and produce a DENSE, anchor-rich CODE_MAP chunk for the "Providers / sports / annotations / utils / tools" subsystem. Use the format: bullet per file/symbol with anchors written as @path::symbol where @path is the repo-relative path EXACTLY as listed (the current layered layout). Capture: key classes/functions/consts; data contracts (mark with §); registries / plugin extension points (mark with ?); and footguns/gotchas (mark with ?). Be precise ? only list symbols that actually exist in the files. If a path is a directory (e.g. tests/), enumerate its files and map each test to the subsystem it covers.
Files/paths: backend/providers/__init__.py, backend/providers/base.py, backend/providers/gemini.py, backend/sports/__init__.py, backend/sports/base.py, backend/sports/badminton.py, backend/annotations/__init__.py, backend/annotations/base.py, backend/annotations/file_provider.py, backend/annotations/automated_provider.py, backend/utils/video.py, backend/utils/geometry.py, backend/tools/rally_slicer.py

2. assistant (2026-06-07T09:10:33.475Z)

I'll read all the specified files to produce the CODE_MAP chunk.

3. user (2026-06-07T09:10:33.956Z)

```
1       from typing import Dict, Type
2       from backend.providers.base import AIVideoProvider
3       from backend.providers.gemini import GeminiProvider
4
5       class ProviderRegistry:
6           def __init__(self):
7               self._registry: Dict[str, Type[AIVideoProvider]] = {}
8               # Register Gemini provider
9               self.register("gemini", GeminiProvider)
10
11          def register(self, name: str, provider_cls: Type[AIVideoProvider]):
12              self._registry[name] = provider_cls
13
14          def get(self, name: str, api_key: str, model_name: str) -> AIVideoProvider:
15              if name not in self._registry:
16                  raise KeyError(f"AI Provider '{name}' is not registered in the
ProviderRegistry. Available: {list(self._registry.keys())}")
17              return self._registry[name](api_key=api_key, model_name=model_name)
18
19          def list_available(self) -> Dict[str, str]:
20              return {name: cls.__doc__ or f"{name.capitalize()} provider" for name, cls in
self._registry.items()}
21
22      provider_registry = ProviderRegistry()
```

```
23
```

4. user (2026-06-07T09:10:34.425Z)

```
1       from abc import ABC, abstractmethod
2       from typing import Tuple
3
4       class AIVideoProvider(ABC):
5           def __init__(self, api_key: str, model_name: str):
6               self.api_key = api_key
7               self.model_name = model_name
8
9           @abstractmethod
10          def analyze_video(self, video_path: str, prompt: str,
11                            chunk_duration: float = 0.0,
12                            label: str = "") -> Tuple[list, dict]:
13              """Sends a video file to the AI model along with a prompt, and returns a tuple
of
14              (segments_list, metrics_dict).
15
16              Args:
17                  video_path: Path to the video file to analyze.
18                  prompt: Prompt instructions.
19                  chunk_duration: Duration of this chunk in seconds, or 0.0 if not chunked.
20                  label: Diagnostic label for logging.
21
22              Returns:
23                  Tuple containing:
24                    - A list of detected segment dicts (e.g. [{"start_time": ..., "end_time":
...}])
25                    - A dictionary of performance metrics (e.g. {"upload_time": ...,
"process_time": ..., "gen_time": ...})
26              """
27              pass
28
```

5. user (2026-06-07T09:10:35.356Z)

```
1       import os
2       import time
3       import json
4       import logging
5       from typing import Tuple
6       from google import genai
7       from google.genai import types
8       from backend.providers.base import AIVideoProvider
9
10      logger = logging.getLogger(__name__)
11
12      # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
13      # each uploaded clip is below this. Oversize uploads are refused (not silently
attempted).
14      MAX_UPLOAD_MB = 500
15
16      class GeminiProvider(AIVideoProvider):
17          def __init__(self, api_key: str, model_name: str):
18              super().__init__(api_key, model_name)
19              # Initialize client lazily to avoid connection/auth issues during class
instantiation
20              self._client = None
21
22          @property
23          def client(self):
24              if self._client is None:
25                  self._client = genai.Client(api_key=self.api_key)
```

```
26              return self._client
27
28          def analyze_video(self, video_path: str, prompt: str,
29                            chunk_duration: float = 0.0,
30                            label: str = "") -> Tuple[list, dict]:
31              log_prefix = f"Gemini {label}" if label else "Gemini"
32
33              metrics = {
34                  "upload_time": 0.0,
35                  "process_time": 0.0,
36                  "gen_time": 0.0
37              }
38
39              # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
40              try:
41                  size_mb = os.path.getsize(video_path) / (1024 * 1024)
42              except OSError:
43                  size_mb = 0.0
44              if size_mb > MAX_UPLOAD_MB:
45                  logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? "
46                               f"chunk smaller. Skipping upload.")
47                  return [], metrics
48
49              # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
50              logger.info(f"[{log_prefix}] Uploading {video_path}...")
51              t_upload_start = time.time()
52              video_file = None
53              for attempt in range(3):
54                  try:
55                      video_file = self.client.files.upload(file=video_path)
56                      metrics["upload_time"] = time.time() - t_upload_start
57                      logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
58                      break
59                  except Exception as e:
60                      logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
61                      if attempt < 2:
62                          time.sleep(2 * (attempt + 1))
63              if video_file is None:
64                  logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
65                  return [], metrics
66
67              # 2. Wait for processing on Google servers
68              t_process_start = time.time()
69              try:
70                  while video_file.state.name == "PROCESSING":
71                      logger.info(f"[{log_prefix}] Processing on Google servers. Elapsed:
{time.time()-t_process_start:.1f}s. Retrying in 10s...")
72                      time.sleep(10)
73                      video_file = self.client.files.get(name=video_file.name)
74                  if video_file.state.name == "FAILED":
75                      logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
76                      try:
77                          self.client.files.delete(name=video_file.name)
78                      except Exception:
79                          pass
80                      return [], metrics
81              except Exception as e:
82                  logger.error(f"[{log_prefix}] Status check failed: {e}")
83                  try:
```

```
84                      self.client.files.delete(name=video_file.name)
85                  except Exception:
86                      pass
87                  return [], metrics
88
89              metrics["process_time"] = time.time() - t_process_start
90              logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
91
92              # 3. Call Gemini (retry transient 503/UNAVAILABLE model-overload + network
blips)
93              logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
94              t_gen_start = time.time()
95              raw_text = None
96              for attempt in range(3):
97                  try:
98                      response = self.client.models.generate_content(
99                          model=self.model_name,
100                         contents=[video_file, prompt],
101                         config=types.GenerateContentConfig(
102                             response_mime_type="application/json",
103                             temperature=0.0
104                         )
105                     )
106                     raw_text = response.text
107                     metrics["gen_time"] = time.time() - t_gen_start
108                     logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
109                     break
110                 except Exception as e:
111                     logger.warning(f"[{log_prefix}] generate attempt {attempt + 1}/3 failed:
{e}")
112                     if attempt < 2:
113                         time.sleep(3 * (attempt + 1))
114             try:
115                 self.client.files.delete(name=video_file.name)
116             except Exception:
117                 pass
118
119             if not raw_text:
120                 return [], metrics
121
122             # 4. Parse JSON
123             try:
124                 segments = json.loads(raw_text)
125                 if not isinstance(segments, list):
126                     logger.error(f"[{log_prefix}] Response is not a JSON array.")
127                     return [], metrics
128                 return segments, metrics
129             except Exception as e:
130                 logger.error(f"[{log_prefix}] JSON parse failed: {e}")
131                 return [], metrics
132
```

6. user (2026-06-07T09:10:35.844Z)

```
1       from typing import Dict, Type
2       from backend.sports.base import SportProfile
3       from backend.sports.badminton import BadmintonSportProfile
4
5       class SportRegistry:
6           def __init__(self):
7               self._registry: Dict[str, Type[SportProfile]] = {}
8               # Automatically register Badminton
```

```
9               self.register(BadmintonSportProfile)
10
11          def register(self, profile_cls: Type[SportProfile]):
12              inst = profile_cls()
13              self._registry[inst.name] = profile_cls
14
15          def get(self, name: str) -> SportProfile:
16              if name not in self._registry:
17                  raise KeyError(f"Sport '{name}' is not registered in the SportRegistry.
Available: {list(self._registry.keys())}")
18              return self._registry[name]()
19
20          def list_available(self) -> Dict[str, str]:
21              return {name: cls.__doc__ or f"{name.capitalize()} sport profile" for name, cls
in self._registry.items()}
22
23      sport_registry = SportRegistry()
24
```

7. user (2026-06-07T09:10:36.315Z)

```
1       from abc import ABC, abstractmethod
2
3       class SportProfile(ABC):
4           @property
5           @abstractmethod
6           def name(self) -> str:
7               """The identifier of the sport (e.g., 'badminton')."""
8               pass
9
10          @abstractmethod
11          def get_prompt(self, chunk_duration: float = 0.0) -> str:
12              """Full system prompt for the AI model, including output schema."""
13              pass
14
15          @abstractmethod
16          def get_relevance_config(self) -> dict:
17              """Sport-specific visual validation params (court colors, thresholds, etc.)."""
18              pass
19
```

8. user (2026-06-07T09:10:36.806Z)

```
1       from backend.sports.base import SportProfile
2
3       class BadmintonSportProfile(SportProfile):
4           @property
5           def name(self) -> str:
6               return "badminton"
7
8           def get_prompt(self, chunk_duration: float = 0.0) -> str:
9               base_prompt = """System Role & Objective
10      You are a highly precise computer vision data extraction engine specializing in
badminton match analysis. Your exclusive objective is to parse a full-length match video and
generate a flawless JSON array containing exact rally boundaries and shot metrics for automated
highlight generation.
11
12      Core Directives
13      Detect every single active rally in the video. Do not miss any live gameplay.
14      Extract high-precision timestamps as TOTAL DECIMAL SECONDS from the start of the video
for exact frame-accurate cuts.
15      Estimate the number of shots exchanged (net crossings) and provide a confidence score
for that count, acknowledging frame-sampling limitations.
16
17      CRITICAL TIMESTAMP FORMAT RULES:
```

```
      - All start_time and end_time values MUST be expressed as TOTAL SECONDS from the beginning of the video, as a decimal number.
      - Example: A rally starting at 1 minute 14 seconds into the video MUST be written as 74.0, NOT as "1:14" or 114.
      - Example: A rally starting at 2 minutes 30.5 seconds MUST be written as 150.5, NOT as "2:30.5" or 230.5.
      - NEVER use MM:SS or HH:MM:SS colon-separated format. NEVER concatenate minutes and seconds into a single number without converting.
      - The downstream pipeline will BREAK if timestamps are not in total decimal seconds.

      Positive Constraints: Rally Boundaries
      Start Condition: The exact fractional second the server's racket makes contact with the shuttlecock.
      End Condition: The rally concludes ONLY when both of the following physical criteria are met sequentially: First, the shuttlecock hits the court surface, strikes the net and drops, or lands visibly out of bounds. Second, a 0.3 to 0.5-second buffer has elapsed where both players have visibly ceased all kinetic momentum (no lunging, sliding, or racket follow-through).

      Negative Constraints: Anomaly Rejection (CRITICAL)
      Replay Rejection: You must strictly ignore and filter out all slow-motion instant replays. Replays are not live rallies.
      Dead-Time Rejection: Ignore all Hawkeye/line-call challenges, interval coaching, towel breaks, court mopping, changing shuttles, and player celebrations.
      Pre-serve Rituals: Do not start the timestamp during the players' setup or stance. Wait for physical racket-to-shuttle contact.

      Shot Tracking & Data Confidence
      Shot Definition: Count every successful traverse of the shuttlecock across the net (serves, clears, smashes, drops, drives). Do not count the final grounding or pre-net faults.
      Confidence Scoring: Because the shuttlecock moves at extreme velocities that may fall between sampled frames, provide a shot_count_confidence metric (Low, Medium, High). Base your estimate on player biomechanics, positioning, and racket swings if the shuttle itself is blurred.

      Strict Output Schema
      Output strictly in the following JSON array format. Do not include markdown formatting, introductory text, conversational filler, or explanations. Any deviation from this schema will break the downstream pipeline.
      Remember: start_time and end_time are TOTAL SECONDS (decimal), not MM:SS.

      [
      {
      "start_time": 74.0,
      "end_time": 81.3,
      "shuttle_exchange_count": 11,
      "shot_count_confidence": "High"
      },
      {
      "start_time": 150.5,
      "end_time": 174.2,
      "shuttle_exchange_count": 24,
      "shot_count_confidence": "Medium"
      }
      ]
      """
              if chunk_duration > 0.0:
                  duration_context = (
                      f"IMPORTANT VIDEO CONTEXT: This video segment is exactly {chunk_duration:.1f} seconds long. "
                      f"All timestamps MUST be between 0 and {chunk_duration:.1f}. "
                      f"Any timestamp beyond {chunk_duration:.1f}s is INVALID.\n\n"
                  )
                  return duration_context + base_prompt
              return base_prompt

```

```
65          def get_relevance_config(self) -> dict:
66              return {
67                  "court_green_lower": [35, 40, 40],
68                  "court_green_upper": [85, 255, 255],
69                  "court_pixels_min_ratio": 0.05
70              }
71
```

9. user (2026-06-07T09:10:37.776Z)

```
1       """Pluggable sources of ground-truth rally annotations.
2
3       `AnnotationProvider`s yield `(start, end)` intervals ? the same shape the eval
4       harness scores ? so golden-set labels can come from a file, a human vendor, or an
5       automated oracle behind one interface. See `docs/GOLDEN_SET_VENDORING.md`.
6       """
7       from .base import annotation_registry, AnnotationProvider
8       from . import file_provider       # noqa: F401  (side-effect: registers FileProvider)
9       from . import automated_provider  # noqa: F401  (side-effect: registers
AutomatedProvider)
10
11      __all__ = ["annotation_registry", "AnnotationProvider"]
12
```

10. user (2026-06-07T09:10:38.258Z)

```
1       """Annotation provider registry ? pluggable sources of ground-truth rally labels.
2
3       An `AnnotationProvider` turns a video reference into a list of `(start, end)` rally
4       intervals ? the **same shape** that `backend.eval.annotations.load_annotations`
5       produces and `backend.eval.metrics` consumes. That symmetry is the whole point:
6       golden-set ground truth can come from a CSV file, a human annotation vendor, or an
7       automated oracle behind one interface, so the regression flow
8       (`backend.eval.regression`) never needs to know where labels came from.
9
10      Mirrors the existing registry pattern in `backend/providers/base.py` and
11      `backend/indexer/base.py`. See `docs/GOLDEN_SET_VENDORING.md` (and ADR-006) for the
12      design rationale.
13      """
14      from abc import ABC, abstractmethod
15      from typing import Dict, List, Optional, Tuple, Type
16
17      # An (start, end) interval in seconds ? identical to backend.eval.metrics.Interval.
18      Interval = Tuple[float, float]
19
20      # Job lifecycle statuses returned by poll().
21      QUEUED = "queued"
22      RUNNING = "running"
23      DONE = "done"
24      FAILED = "failed"
25      STATUSES = (QUEUED, RUNNING, DONE, FAILED)
26
27
28      class AnnotationProvider(ABC):
29          """Abstract source of ground-truth rally intervals for a video.
30
31          The three-method contract (submit ? poll ? fetch) accommodates both
32          synchronous providers (a file already on disk) and asynchronous ones (a human
33          vendor that takes days). Synchronous providers can use the `annotate()`
34          convenience, which runs the full cycle in one call.
35          """
36
37          @property
38          @abstractmethod
39          def name(self) -> str:
```

```
            """Unique registry key for this provider (e.g. 'file', 'vendor', 'auto')."""

        @abstractmethod
        def submit(self, video_ref: str, guideline: Optional[str] = None,
                   sport: str = "badminton") -> str:
            """Submit a video for annotation; return an opaque job_id.

            `video_ref` is provider-defined (a file path, a video_id/MD5, a URL).
            `guideline` is the annotation spec (what counts as a rally start/end).
            """

        @abstractmethod
        def poll(self, job_id: str) -> str:
            """Return one of STATUSES for the given job."""

        @abstractmethod
        def fetch(self, job_id: str) -> List[Interval]:
            """Return the completed annotations as sorted `(start, end)` intervals.

            Should raise if the job is not DONE / the labels are unavailable, rather
            than returning an empty list (which would silently skew downstream scores).
            """

        def annotate(self, video_ref: str, guideline: Optional[str] = None,
                     sport: str = "badminton") -> List[Interval]:
            """Convenience: submit + fetch for a provider that completes synchronously.

            Async providers (e.g. a human vendor with a multi-day SLA) should be driven
            via submit/poll/fetch explicitly instead ? calling this on one will raise
            once the job isn't immediately DONE.
            """
            job_id = self.submit(video_ref, guideline=guideline, sport=sport)
            status = self.poll(job_id)
            if status != DONE:
                raise RuntimeError(
                    f"{self.name}.annotate(): job '{job_id}' is '{status}', not DONE. "
                    f"Use submit/poll/fetch for asynchronous providers."
                )
            return self.fetch(job_id)


    class AnnotationProviderRegistry:
        """Registry of annotation providers, keyed by each provider's `name`."""

        def __init__(self):
            self._providers: Dict[str, Type[AnnotationProvider]] = {}

        def register(self, provider_cls: Type[AnnotationProvider]) ->
Type[AnnotationProvider]:
            """Register a provider class under its `name`. Returns the class (decorator-
friendly)."""
            try:
                key = provider_cls().name
            except Exception:
                # Providers needing constructor args fall back to the class name.
                key = getattr(provider_cls, "__name__", "unknown")
            self._providers[key] = provider_cls
            return provider_cls

        def get(self, name: str, **kwargs) -> AnnotationProvider:
            """Instantiate the named provider, passing through any constructor kwargs."""
            cls = self._providers.get(name)
            if cls is None:
                raise KeyError(
                    f"Unknown annotation provider '{name}'. Available:
```

```
{self.list_available()}"
                )
            return cls(**kwargs)

        def list_available(self) -> List[str]:
            return list(self._providers.keys())


    annotation_registry = AnnotationProviderRegistry()

```

11. user (2026-06-07T09:10:38.725Z)

```
      """FileProvider ? ground-truth rally labels from pre-made CSV files.

      This formalizes today's manual / ShuttleSet annotation flow behind the
      `AnnotationProvider` interface: the "annotation job" is simply a CSV that already
      exists on disk (the same `start,end` format `run_eval.py --scaffold` writes). The
      job_id IS the resolved CSV path, so submit/poll/fetch all operate on that path.

      It's the synchronous, zero-cost provider used as the default golden-set source and
      as the stand-in in tests/CI before a human-vendor or automated provider is wired in.
      """
      import os
      from typing import List, Optional

      from backend.annotations.base import (
          AnnotationProvider,
          annotation_registry,
          Interval,
          DONE,
          FAILED,
      )
      from backend.eval.annotations import load_annotations


      class FileProvider(AnnotationProvider):
          def __init__(self, csv_dir: Optional[str] = None, ext: str = ".csv"):
              """
              Args:
                  csv_dir: if set, a bare `video_ref` (not itself a path) is resolved to
                           `<csv_dir>/<video_ref><ext>` ? handy for keying by video_id/MD5.
                  ext: file extension used for that directory-based resolution.
              """
              self.csv_dir = csv_dir
              self.ext = ext

          @property
          def name(self) -> str:
              return "file"

          def _resolve(self, video_ref: str) -> str:
              """Map a video_ref to a CSV path.

              An existing path is used as-is; otherwise it's treated as a video_id and
              looked up under csv_dir (if configured).
              """
              if os.path.isfile(video_ref):
                  return video_ref
              if self.csv_dir:
                  return os.path.join(self.csv_dir, f"{video_ref}{self.ext}")
              return video_ref

          def submit(self, video_ref: str, guideline: Optional[str] = None,
                     sport: str = "badminton") -> str:
```

```
            # Nothing to do ? the label file is expected to already exist. The job_id
            # is the resolved CSV path. (guideline/sport are irrelevant for a file.)
            return self._resolve(video_ref)

        def poll(self, job_id: str) -> str:
            return DONE if os.path.isfile(job_id) else FAILED

        def fetch(self, job_id: str) -> List[Interval]:
            if not os.path.isfile(job_id):
                raise FileNotFoundError(f"annotation file not found: {job_id}")
            return load_annotations(job_id)


    annotation_registry.register(FileProvider)

```

12. user (2026-06-07T09:10:39.630Z)

```
    """AutomatedProvider ? the fast-tier "oracle" annotation source (GS-4, interface stub).

    The fast/CI tier of the golden-set flow needs an *automated* labeler that turns a
    video into rally `(start, end)` intervals quickly and cheaply, as a smoke alarm
    between authoritative human-vendor runs (the slow tier).

    This slice ships the **interface only**, deliberately model-agnostic: an
    `AutomatedProvider` that delegates labeling to an injected `labeler` callable, plus
    a deterministic fake for tests/CI. No concrete model is wired yet ? that's a
    tracked backlog item (docs/BACKLOG.md), to be chosen alongside the production LLM
    step so the lineages differ (see the oracle rule below).

    ?? **Oracle rule.** The labeler MUST be a *different model lineage* than whatever
    production uses. If production becomes the offline ActionFormer segmenter and the
    oracle is also ActionFormer, they share blind spots and the regression test is
    blind ? green while both are wrong. `lineage` records the oracle's model family so
    callers can guard against it matching production (`warn_if_same_lineage`).
    """
    import logging
    from typing import Callable, Dict, List, Optional

    from backend.annotations.base import (
        AnnotationProvider,
        annotation_registry,
        Interval,
        DONE,
        FAILED,
    )

    logger = logging.getLogger(__name__)

    # A labeler turns (video_ref, guideline, sport) into rally intervals.
    Labeler = Callable[[str, Optional[str], str], List[Interval]]


    def _not_configured(video_ref: str, guideline: Optional[str], sport: str) ->
List[Interval]:
        """Default labeler: refuses to run until a real model is injected.

        Keeps `AutomatedProvider()` constructible (so the registry can key it as 'auto')
        while making accidental use of an unconfigured oracle fail loudly rather than
        silently returning no rallies.
        """
        raise NotImplementedError(
            "AutomatedProvider has no labeler configured. Inject one via "
            "annotation_registry.get('auto', labeler=<fn>, lineage='<model-family>'). "
            "Choosing the concrete oracle model is a tracked item ? see docs/BACKLOG.md
```

```
(GS-4)."
        )


    def warn_if_same_lineage(oracle_lineage: Optional[str], production_lineage:
Optional[str]) -> bool:
        """Warn (and return True) if the oracle shares production's model lineage.

        Encodes the oracle rule in code, not just docs: a same-lineage oracle can't be
        trusted to catch production's failures. Callers wiring CI should pass both.
        """
        if oracle_lineage and production_lineage and oracle_lineage == production_lineage:
            logger.warning(
                "Oracle lineage '%s' matches production's ? the regression test may be "
                "blind to shared failure modes. Use a different model lineage.",
                oracle_lineage,
            )
            return True
        return False


    class AutomatedProvider(AnnotationProvider):
        def __init__(self, labeler: Optional[Labeler] = None, lineage: str = "unspecified"):
            """
            Args:
                labeler: callable producing rally intervals for a video. Defaults to a
                         stub that raises until configured.
                lineage: the oracle's model family (e.g. 'gemini', 'qwen-vl') ? recorded
                         so callers can enforce the oracle rule via warn_if_same_lineage.
            """
            self._labeler = labeler or _not_configured
            self.lineage = lineage
            # In-process synchronous job store: job_id -> labeled intervals.
            self._jobs: Dict[str, List[Interval]] = {}

        @property
        def name(self) -> str:
            return "auto"

        def submit(self, video_ref: str, guideline: Optional[str] = None,
                   sport: str = "badminton") -> str:
            # Synchronous stub: run the labeler now and stash the result. A real async
            # oracle (e.g. a queued API job) would kick off here and resolve in poll().
            intervals = self._labeler(video_ref, guideline, sport)
            intervals = sorted((float(s), float(e)) for s, e in intervals)
            job_id = video_ref
            self._jobs[job_id] = intervals
            return job_id

        def poll(self, job_id: str) -> str:
            return DONE if job_id in self._jobs else FAILED

        def fetch(self, job_id: str) -> List[Interval]:
            if job_id not in self._jobs:
                raise KeyError(f"AutomatedProvider: unknown job '{job_id}' (submit it
first)")
            return self._jobs[job_id]


    annotation_registry.register(AutomatedProvider)

```

13. user (2026-06-07T09:10:40.132Z)

```
    import subprocess
```

```
2       import os
3       import shutil
4       import logging
5
6       logger = logging.getLogger(__name__)
7
8       def get_video_duration(video_path: str) -> float:
9           """Returns the exact duration of a video file in seconds using ffprobe.
10          Returns 0.0 on failure."""
11          try:
12              duration_output = subprocess.check_output([
13                  "ffprobe", "-v", "error", "-show_entries", "format=duration",
14                  "-of", "default=noprint_wrappers=1:nokey=1", video_path
15              ]).decode().strip()
16              return float(duration_output)
17          except Exception as e:
18              logger.error(f"Could not detect video duration for {video_path}: {e}")
19              return 0.0
20
21      def extract_video_chunk(video_path: str, start_time: float, duration: float,
22                              output_path: str, reencode: bool = False,
23                              scale_width: int = None) -> bool:
24          """Extracts a chunk from a video file.
25
26          Args:
27              reencode: If True, re-encodes with libx264 (slower but frame-accurate).
28                        If False, uses stream-copy (fast but cuts at nearest keyframe).
29                        Use True for short clips where keyframe drift matters.
30              scale_width: If set (requires reencode), downscale to this width (height auto,
31                        even). Useful for high-res sources headed to an API that analyses at
32                        low res anyway ? shrinks uploads with no analysis loss.
33          """
34          cmd = [
35              "ffmpeg", "-y",
36              "-ss", str(start_time),
37              "-t", str(duration),
38              "-i", video_path,
39          ]
40          if reencode:
41              if scale_width:
42                  cmd += ["-vf", f"scale={int(scale_width)}:-2"]
43              cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
44          else:
45              cmd += ["-c", "copy"]
46          cmd.append(output_path)
47          try:
48              subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
49              return True
50          except Exception as e:
51              logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s:
{e}")
52              return False
53
```

14. user (2026-06-07T09:10:40.613Z)

```
1       import math
2       from typing import Tuple
3
4       def get_center(bbox: Tuple[float, float, float, float]) -> Tuple[float, float]:
5           """Calculate the center (x, y) of a bounding box.
6           bbox format: (x1, y1, x2, y2)
7           """
8           x1, y1, x2, y2 = bbox
```

```
9           return ((x1 + x2) / 2.0, (y1 + y2) / 2.0)
10
11      def get_distance(p1: Tuple[float, float], p2: Tuple[float, float]) -> float:
12          """Calculate the Euclidean distance between two points (x, y)."""
13          return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)
14
15      def get_velocity(bbox1: Tuple[float, float, float, float], bbox2: Tuple[float, float,
float, float], fps: float) -> float:
16          """Calculate velocity (pixels per second) of a bounding box moving between two
consecutive frames.
17          """
18          c1 = get_center(bbox1)
19          c2 = get_center(bbox2)
20          dist = get_distance(c1, c2)
21          return dist * fps
22
23      def get_velocity_normalised(bbox1: Tuple[float, float, float, float], bbox2:
Tuple[float, float, float, float],
24                                  fps: float, frame_width: float) -> float:
25          """Calculate velocity as a fraction of frame width per second.
26
27          This makes the threshold resolution-independent:
28          - 0.15 means the center moved 15% of the frame width in one second.
29          - Works identically on 720p, 1080p, and 4K footage.
30
31          Args:
32              bbox1: Previous bounding box (x1, y1, x2, y2).
33              bbox2: Current bounding box (x1, y1, x2, y2).
34              fps: Effective frames per second (after frame-skipping).
35              frame_width: Width of the video frame in pixels.
36          """
37          raw_velocity = get_velocity(bbox1, bbox2, fps)
38          if frame_width <= 0:
39              return raw_velocity  # Fallback to raw pixels if width unknown
40          return raw_velocity / frame_width
41
42      def calculate_iou(boxA: Tuple[float, float, float, float], boxB: Tuple[float, float,
float, float]) -> float:
43          """Calculate the Intersection over Union (IoU) of two bounding boxes.
44          bbox format: (x1, y1, x2, y2)
45          """
46          xA = max(boxA[0], boxB[0])
47          yA = max(boxA[1], boxB[1])
48          xB = min(boxA[2], boxB[2])
49          yB = min(boxA[3], boxB[3])
50
51          interArea = max(0, xB - xA + 1) * max(0, yB - yA + 1)
52          if interArea == 0:
53              return 0.0
54
55          boxAArea = (boxA[2] - boxA[0] + 1) * (boxA[3] - boxA[1] + 1)
56          boxBArea = (boxB[2] - boxB[0] + 1) * (boxB[3] - boxB[1] + 1)
57
58          iou = interArea / float(boxAArea + boxBArea - interArea)
59          return iou
60
```

15. user (2026-06-07T09:10:41.053Z)

```
1       """Slice individual rally clips out of a (long) annotated video.
2
3       Use cases:
4       - **Offline verification** ? pull rally N out of a 30-min match to eyeball its
5         boundaries against the labels.
6       - **Per-video calibration/training** ? grab a handful of labeled rallies as the
```

```
  supervision the segmenter is tuned/adapted on for that specific video.

Given a labels CSV (the golden `rally_number,start_time,end_time[,start_mmss,
end_mmss]` shape) and a set of rally numbers, it writes one padded clip per rally:
    <out_dir>/<video_stem>__rally_<n>.mp4

Padding matches the highlight compiler (begin 0.5 s / end 1.0 s) so a sliced clip
shows the same lead-in/out the model is trained and evaluated against. The window
math (`compute_clip_windows`) is pure and unit-tested; ffmpeg is only touched in
`slice_rallies`.
"""
from __future__ import annotations

import argparse
import csv
import os
import subprocess
from dataclasses import dataclass
from typing import Dict, List, Optional, Tuple

from backend.eval.annotations import parse_timestamp

# Match VideoCompiler defaults so sliced clips frame a rally the same way.
BEGIN_PADDING = 0.5
END_PADDING = 1.0


@dataclass
class ClipWindow:
    rally: str          # kept as a string to allow inserted ids like "7.5"
    start: float        # padded, clamped
    end: float          # padded, clamped


def _read_time(row: Dict[str, str], decimal_key: str, mmss_key: str) -> Optional[float]:
    """Prefer the decimal-seconds column; fall back to mm:ss. None if unusable."""
    for key in (decimal_key, mmss_key):
        val = (row.get(key) or "").strip()
        if val:
            try:
                return parse_timestamp(val)
            except ValueError:
                continue
    return None


def load_rally_labels(csv_path: str) -> Dict[str, Tuple[float, float]]:
    """Map rally_number -> (start, end) seconds, skipping rows with no usable times.

    Accepts the golden schema (start_time/end_time decimal, optional start_mmss/
    end_mmss). Rally numbers are kept as strings so inserted ids ("7.5") survive.
    """
    labels: Dict[str, Tuple[float, float]] = {}
    with open(csv_path, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[])]
        for row in reader:
            row = {(k or "").strip().lower(): v for k, v in row.items()}
            rally = (row.get("rally_number") or "").strip()
            if not rally:
                continue
            start = _read_time(row, "start_time", "start_mmss")
            end = _read_time(row, "end_time", "end_mmss")
            if start is None or end is None or end <= start:
```

```
                    continue  # missing/degenerate (e.g. a not-yet-filled MISSED row)
                labels[rally] = (start, end)
        return labels


    def compute_clip_windows(labels: Dict[str, Tuple[float, float]],
                             rally_numbers: List[str],
                             begin_pad: float = BEGIN_PADDING,
                             end_pad: float = END_PADDING,
                             video_duration: Optional[float] = None) -> List[ClipWindow]:
        """Apply padding and clamp to the video, for the requested rallies.

        rally_numbers may be ["all"] to take every labeled rally (in time order).
        Unknown rally ids are skipped (caller can diff to warn).
        """
        if len(rally_numbers) == 1 and rally_numbers[0].lower() == "all":
            wanted = sorted(labels, key=lambda r: labels[r][0])
        else:
            wanted = [r for r in rally_numbers if r in labels]

        windows: List[ClipWindow] = []
        for rally in wanted:
            start, end = labels[rally]
            ps = max(0.0, start - begin_pad)
            pe = end + end_pad
            if video_duration is not None and video_duration > 0:
                if ps >= video_duration:
                    continue  # entirely past the end
                pe = min(pe, video_duration)
            if pe <= ps:
                continue
            windows.append(ClipWindow(rally=rally, start=round(ps, 3), end=round(pe, 3)))
        return windows


    def _probe_duration(video_path: str) -> Optional[float]:
        try:
            out = subprocess.run(
                ["ffprobe", "-v", "error", "-show_entries", "format=duration",
                 "-of", "default=noprint_wrappers=1:nokey=1", video_path],
                capture_output=True, text=True, check=True,
            ).stdout.strip()
            return float(out)
        except (subprocess.SubprocessError, OSError, ValueError):
            return None


    def slice_rallies(video_path: str, labels_csv: str, rally_numbers: List[str],
                      out_dir: str, begin_pad: float = BEGIN_PADDING,
                      end_pad: float = END_PADDING) -> List[str]:
        """Write one padded clip per requested rally; returns the clip paths written."""
        if not os.path.exists(video_path):
            raise FileNotFoundError(f"video not found: {video_path}")
        labels = load_rally_labels(labels_csv)
        if not labels:
            raise ValueError(f"no usable rally rows in {labels_csv}")
        os.makedirs(out_dir, exist_ok=True)

        duration = _probe_duration(video_path)
        windows = compute_clip_windows(labels, rally_numbers, begin_pad, end_pad, duration)
        if not windows:
            raise ValueError("no rallies matched the request (check --rallies vs the CSV)")

        stem = os.path.splitext(os.path.basename(video_path))[0]
        written: List[str] = []
```

```
        for w in windows:
            safe_rally = w.rally.replace(".", "_")
            out_path = os.path.join(out_dir, f"{stem}__rally_{safe_rally}.mp4")
            cmd = [
                "ffmpeg", "-y",
                "-ss", str(w.start), "-to", str(w.end), "-i", video_path,
                "-c:v", "libx264", "-preset", "superfast", "-crf", "22",
                "-c:a", "aac", "-b:a", "128k",
                out_path,
            ]
            try:
                subprocess.run(cmd, capture_output=True, check=True)
            except subprocess.CalledProcessError as e:
                err = e.stderr.decode(errors="replace").strip()[-300:]
                print(f"[slicer] rally {w.rally}: ffmpeg failed: {err}")
                continue
            if os.path.exists(out_path) and os.path.getsize(out_path) > 0:
                written.append(out_path)
                print(f"[slicer] rally {w.rally} -> {out_path}
[{w.start:.3f}-{w.end:.3f}s]")
        return written


    def main() -> None:
        p = argparse.ArgumentParser(description="Slice rally clips from an annotated
video.")
        p.add_argument("--video", required=True, help="Source video file")
        p.add_argument("--labels", required=True, help="Labels CSV (golden rally format)")
        p.add_argument("--rallies", required=True,
                       help="Comma-separated rally numbers (e.g. 7,7.5,16) or 'all'")
        p.add_argument("--out-dir", default="output/rally_clips", help="Output directory")
        p.add_argument("--begin-pad", type=float, default=BEGIN_PADDING)
        p.add_argument("--end-pad", type=float, default=END_PADDING)
        args = p.parse_args()

        rally_numbers = [r.strip() for r in args.rallies.split(",") if r.strip()]
        written = slice_rallies(args.video, args.labels, rally_numbers,
                                args.out_dir, args.begin_pad, args.end_pad)
        print(f"\nWrote {len(written)} clip(s) to {args.out_dir}")


    if __name__ == "__main__":
        main()

```

16. assistant (2026-06-07T09:11:47.832Z)

All files read. Producing the CODE_MAP chunk.

Subsystem: Providers / Sports / Annotations / Utils / Tools

Three parallel registry-based plugin subsystems (AI providers, sport profiles, annotation sources) plus stateless utility/tool modules. All three registries follow the same pattern: a `base.py` ABC + a registry instance auto-populated at import time.

AI Providers ? `backend/providers/` (LLM video-analysis backends)

- ? `@backend/providers/__init__.py::ProviderRegistry` ? registry keyed by string name ? `Type[AIVideoProvider]`; module-level singleton `@backend/providers/__init__.py::provider_registry`. Auto-registers `"gemini"` in `__init__`.
  - `ProviderRegistry.register(name, provider_cls)` ? manual registration (NOT decorator-style; differs from annotation registry).
  - `ProviderRegistry.get(name, api_key, model_name)` ? instantiates with creds; raises

`KeyError` listing available names if missing.
  - `ProviderRegistry.list_available()` ? `{name: docstring}`.
- `@backend/providers/base.py::AIVideoProvider` ? ABC; ctor takes `(api_key, model_name)`.
  - § `@backend/providers/base.py::AIVideoProvider.analyze_video(video_path, prompt, chunk_duration=0.0, label="")` ? **`Tuple[list, dict]`** = `(segments_list, metrics_dict)`. Metrics keys: `upload_time`, `process_time`, `gen_time`. Segment dicts shaped like `{"start_time", "end_time", ...}`. This tuple contract is the cross-cutting data shape every provider must honor.
- `@backend/providers/gemini.py::GeminiProvider` ? Google `genai` impl (`from google import genai`).
  - `@backend/providers/gemini.py::GeminiProvider.client` ? lazy `@property` building `genai.Client(api_key=...)` on first use (avoids auth at instantiation).
  - § const `@backend/providers/gemini.py::MAX_UPLOAD_MB` = `500` ? soft cap (real Gemini File API limit is 2GB; capped lower for safety/speed).
  - `GeminiProvider.analyze_video(...)` ? pipeline: size-check ? upload (3 retries) ? poll `PROCESSING` state (10s loop) ? `generate_content` (3 retries, `response_mime_type="application/json"`, `temperature=0.0`) ? delete remote file ? `json.loads`.
  - ? On ANY failure (oversize, upload fail, processing FAILED, status check, empty response, non-array JSON, parse error) returns `([], metrics)` ? **empty list, never raises**. Callers cannot distinguish "no rallies" from "provider failed" by return value alone (metrics still populated).
  - ? Requires `ffprobe`/network/`GEMINI` access; uploaded file is always deleted in a best-effort `try/except` (swallowed) ? orphaned remote files possible if process dies mid-run.
  - ? Caller MUST pre-chunk so each clip < `MAX_UPLOAD_MB`; oversize clips are silently skipped (logged error + empty return).

Sports ? `backend/sports/` (per-sport prompts + relevance config)
- ? `@backend/sports/__init__.py::SportRegistry` ? registry keyed by `profile.name` ? `Type[SportProfile]`; singleton `@backend/sports/__init__.py::sport_registry`. Auto-registers `BadmintonSportProfile`.
  - ? `SportRegistry.register(profile_cls)` **instantiates** the profile (`profile_cls()`) just to read `.name` for the key ? profiles must be cheaply constructible with no args.
  - `SportRegistry.get(name)` ? fresh instance; `KeyError` lists available if missing. `list_available()` ? `{name: docstring}`.
- `@backend/sports/base.py::SportProfile` ? ABC. Abstract members: `name` (`@property`), `get_prompt(chunk_duration=0.0)`, `get_relevance_config()`.
- `@backend/sports/badminton.py::BadmintonSportProfile` ? `name == "badminton"`.
  - § `BadmintonSportProfile.get_prompt(chunk_duration=0.0)` ? large hardcoded CV-extraction system prompt. Output schema is a JSON array of `{start_time, end_time, shuttle_exchange_count, shot_count_confidence}` where times are **TOTAL DECIMAL SECONDS** (heavily emphasized; rejects MM:SS / concatenated formats). `shot_count_confidence ? {Low, Medium, High}`. When `chunk_duration > 0.0`, prepends a context block clamping all timestamps to `[0, chunk_duration]`.
  - § `BadmintonSportProfile.get_relevance_config()` ? `{"court_green_lower": [35,40,40], "court_green_upper": [85,255,255], "court_pixels_min_ratio": 0.05}` ? HSV green-court thresholds for downstream visual relevance validation.
  - ? Prompt schema (`shuttle_exchange_count`/`shot_count_confidence`) differs from the provider base's generic `{start_time,end_time}` contract ? provider returns raw model JSON; sport prompt defines the richer keys.

Annotations ? `backend/annotations/` (pluggable ground-truth rally-label sources)
Golden-set vendoring layer (ADR-006, `docs/GOLDEN_SET_VENDORING.md`). Mirrors the provider registry pattern but uses **decorator-friendly self-registration**.
- ? `@backend/annotations/__init__.py` ? re-exports `annotation_registry`, `AnnotationProvider`; imports `file_provider` & `automated_provider` purely for their registration **side-effects** (noqa F401). `__all__ = ["annotation_registry", "AnnotationProvider"]`.
- § `@backend/annotations/base.py::Interval` = `Tuple[float, float]` ? **(start, end) seconds; identical to `backend.eval.metrics.Interval`**. This symmetry is the whole point: ground truth from file/vendor/oracle all feed the eval harness unchanged.
- § Status consts `@backend/annotations/base.py::QUEUED`/`RUNNING`/`DONE`/`FAILED` and tuple `STATUSES` = `("queued","running","done","failed")`.
- `@backend/annotations/base.py::AnnotationProvider` ? ABC, **submit ? poll ? fetch** lifecycle (supports async vendors).
  - § `name` (`@property`, registry key), `submit(video_ref, guideline=None, sport="badminton") ? job_id:str`, `poll(job_id) ? status:str`, `fetch(job_id) ? List[Interval]` (sorted; ? should

raise if not DONE rather than return `[]`, which would silently skew scores).
  - `AnnotationProvider.annotate(video_ref, guideline=None, sport="badminton")` ? convenience submit+poll+fetch for synchronous providers; ? **raises `RuntimeError`** if job isn't immediately `DONE` (async providers must drive submit/poll/fetch manually).
- ? `@backend/annotations/base.py::AnnotationProviderRegistry` ? keyed by provider `name`; singleton `@backend/annotations/base.py::annotation_registry`.
  - `register(provider_cls)` ? returns the class (**decorator-friendly**); keys by `provider_cls().name`, ? falling back to `__name__` if the no-arg ctor throws (providers needing ctor args register under their class name, not their `.name`).
  - `get(name, **kwargs)` ? instantiates passing through ctor kwargs (e.g. `labeler=`, `lineage=`); `KeyError` lists available. `list_available()` ? `List[str]` (note: list of names, NOT the `{name:doc}` dict the provider/sport registries return).
- `@backend/annotations/file_provider.py::FileProvider` ? `name == "file"`. Synchronous, zero-cost default; the "job" is a CSV already on disk (same `start,end` format as `run_eval.py --scaffold`). Self-registers via `annotation_registry.register(FileProvider)` at bottom.
  - ctor `FileProvider(csv_dir=None, ext=".csv")`. `_resolve(video_ref)`: existing path used as-is, else `<csv_dir>/<video_ref><ext>` (key by video_id/MD5). `submit` returns the resolved path **as the job_id**; `poll` ? `DONE`/`FAILED` on `os.path.isfile`; `fetch` delegates to `@backend/eval/annotations::load_annotations` (raises `FileNotFoundError` if missing).
  - ? `guideline`/`sport` args are ignored (a file's labels already exist).
- `@backend/annotations/automated_provider.py::AutomatedProvider` ? `name == "auto"`. **Interface stub only** (GS-4); no concrete model wired. Self-registers at bottom.
  - § type alias `@backend/annotations/automated_provider.py::Labeler` = `Callable[[str, Optional[str], str], List[Interval]]` (= `(video_ref, guideline, sport) ? intervals`).
  - ctor `AutomatedProvider(labeler=None, lineage="unspecified")` ? defaults labeler to `_not_configured`; in-process synchronous job store `_jobs: Dict[job_id, List[Interval]]`. `submit` runs labeler now, sorts `(float,float)` intervals, stashes under `job_id = video_ref`. `poll`/`fetch` read `_jobs`.
  - `@backend/annotations/automated_provider.py::_not_configured(...)` ? default labeler that **raises `NotImplementedError`** (keeps provider constructible for registry keying while failing loudly on accidental use; points to `docs/BACKLOG.md` GS-4).
  - ? ORACLE RULE ? `@backend/annotations/automated_provider.py::warn_if_same_lineage(oracle_lineage, production_lineage)` ? `bool`; logs a warning + returns `True` when oracle and production share a model lineage (shared blind spots make the regression test green-while-wrong). It only **warns**, does not block ? CI wiring must act on the bool. `lineage` ctor arg records the oracle's family for this guard.
  - ? Registered under name `"auto"` only because the no-arg ctor succeeds (labeler defaults to the stub) ? see the registry's `__name__` fallback caveat above.

Utils ? `backend/utils/` (stateless helpers)
- `@backend/utils/video.py` ? ffmpeg/ffprobe wrappers (? require `ffmpeg` + `ffprobe` on PATH).
  - `@backend/utils/video.py::get_video_duration(video_path)` ? float seconds via `ffprobe`; ? returns `0.0` on any failure (logged) ? caller must treat 0.0 as "unknown", not a real duration.
  - `@backend/utils/video.py::extract_video_chunk(video_path, start_time, duration, output_path, reencode=False, scale_width=None)` ? `bool`. ? `reencode=False` = fast stream-copy but cuts at **nearest keyframe** (boundary drift); `reencode=True` (libx264 ultrafast/crf23) = frame-accurate. `scale_width` requires `reencode=True` (else ignored). Returns `False` on failure; ffmpeg stdout/stderr sent to DEVNULL.
- `@backend/utils/geometry.py` ? pure bbox math, format `(x1, y1, x2, y2)`.
  - `get_center(bbox)`, `get_distance(p1, p2)` (Euclidean), `get_velocity(bbox1, bbox2, fps)` (px/s of center), `get_velocity_normalised(bbox1, bbox2, fps, frame_width)` (fraction of frame-width/s ? resolution-independent thresholds; ? falls back to raw px velocity if `frame_width <= 0`).
  - ? `@backend/utils/geometry.py::calculate_iou(boxA, boxB)` uses the `+1` pixel-area convention (`(x2-x1+1)*(y2-y1+1)`); returns `0.0` when intersection area is 0. Mixing with `+0` IoU conventions elsewhere will skew thresholds.

Tools ? `backend/tools/` (standalone CLIs)
- `@backend/tools/rally_slicer.py` ? CLI to cut padded per-rally clips from a long annotated video (offline verification / per-video calibration). Output naming: `<out_dir>/<video_stem>__rally_<n>.mp4` (rally id `.` ? `_`).
  - § consts `@backend/tools/rally_slicer.py::BEGIN_PADDING` = `0.5`, `END_PADDING` = `1.0` ? ? deliberately match `VideoCompiler` defaults so sliced clips frame rallies identically to train/eval.

  - § `@backend/tools/rally_slicer.py::ClipWindow` ? dataclass `(rally: str, start: float, end: float)`; `rally` kept as **string** to allow inserted ids like `"7.5"`.
  - `@backend/tools/rally_slicer.py::load_rally_labels(csv_path)` ? `Dict[str, (start, end)]`. Accepts golden schema `rally_number,start_time,end_time[,start_mmss,end_mmss]`; ? lowercases/strips headers, prefers decimal cols, falls back to mm:ss via `@backend/eval/annotations::parse_timestamp`; **silently skips** rows with missing/degenerate times (`end <= start`, e.g. unfilled MISSED rows).
  - `@backend/tools/rally_slicer.py::compute_clip_windows(labels, rally_numbers, begin_pad, end_pad, video_duration=None)` ? `List[ClipWindow]` ? **pure & unit-tested** window math (pad + clamp to duration). `rally_numbers == ["all"]` ? every labeled rally in time order; unknown ids silently dropped; windows entirely past EOF or with `pe <= ps` skipped; rounds to 3 dp.
  - `@backend/tools/rally_slicer.py::_probe_duration(video_path)` ? `Optional[float]` (ffprobe; `None` on failure ? separate from `utils/video.get_video_duration` which returns `0.0`).
  - `@backend/tools/rally_slicer.py::slice_rallies(video_path, labels_csv, rally_numbers, out_dir, begin_pad, end_pad)` ? `List[str]` written paths. The ONLY ffmpeg-touching fn (libx264 superfast/crf22 + aac 128k, `-ss/-to` accurate cut). ? Raises `FileNotFoundError`/`ValueError` for missing video / no usable rows / no matched rallies; per-rally ffmpeg failures are caught + printed + skipped (not fatal).
  - `@backend/tools/rally_slicer.py::main()` ? argparse entry (`--video --labels --rallies --out-dir --begin-pad --end-pad`); `--rallies` is comma-separated or `all`.

Cross-cutting notes

- ? Three registry idioms diverge: providers/sports use **manual `register()` + `list_available()?dict`**; annotations use **self-registering decorator-friendly `register()` returning the class + `list_available()?list`**. Don't assume one shape.
- § The `(start, end)` / `Interval` seconds tuple is the universal label currency tying `backend/annotations`, `backend/eval/metrics`, and `backend/eval/annotations.load_annotations` together.
- External deps not in this subsystem but referenced: `@backend/eval/annotations::load_annotations` (FileProvider), `@backend/eval/annotations::parse_timestamp` (rally_slicer). System deps: `ffmpeg`/`ffprobe` (video.py, rally_slicer), `google.genai` (gemini).
- Note: no `tests/` directory was in the provided file list, so none enumerated; `compute_clip_windows` and the geometry helpers are referenced as unit-tested but their test files were not in scope.